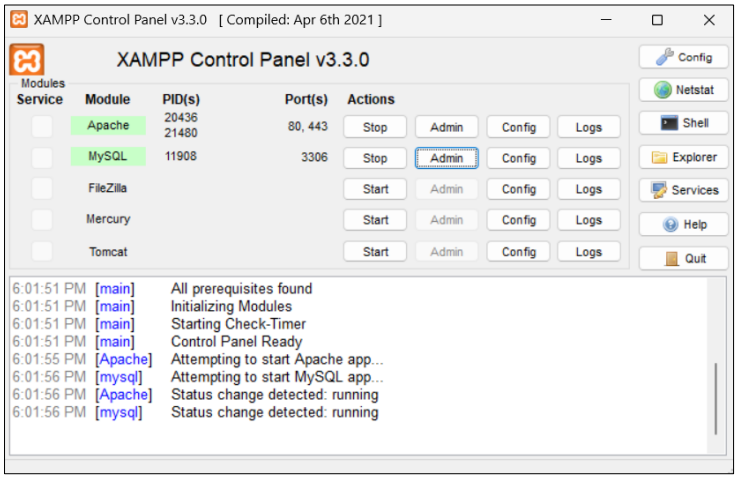
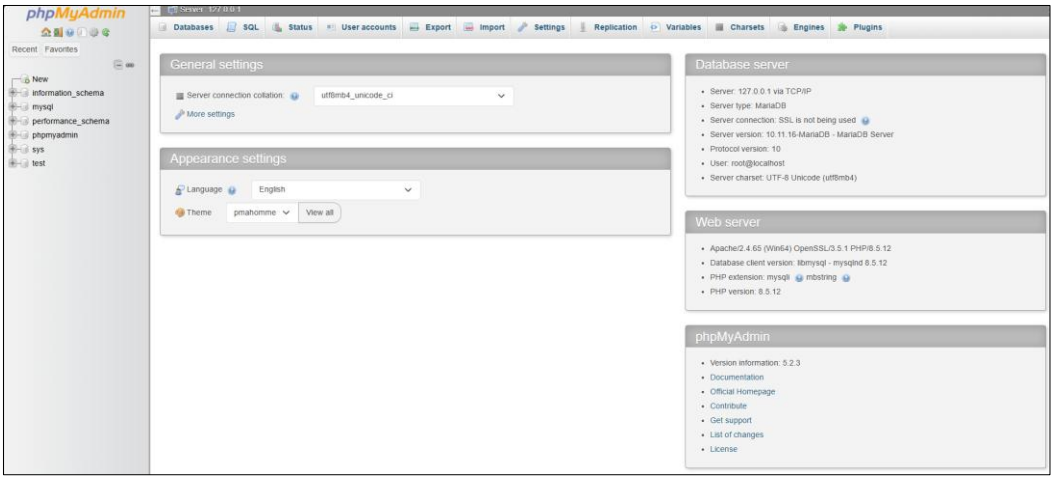
Name: Balleras, Vhon Xander R.	**Date:** October 28-30, 2026
Course: Data Analytics NCIII	**Trainer:** Christian G. Igay

MySQL Activities

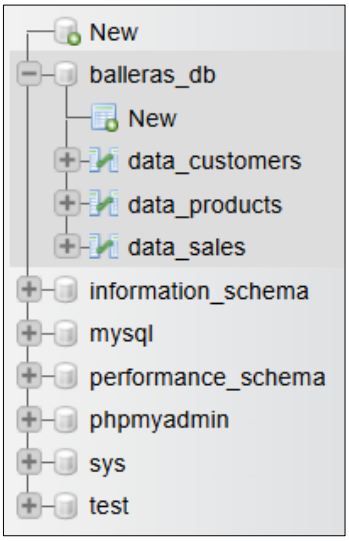




TASKS:

Tables: Sales, Customers, Products

1. Add product, customer, sales.
2. Update product and customer.
3. Delete product and customer.
4. Query customers.
5. Query sales.
6. Query products.
7. Query customers purchases.
8. Query customers total purchase amount.
9. Query Average sales.
10. Query top customers based on sales.
11. Query product sales (specific product).
12. Query the total sales of specific product.
13. Query the average sales of specific product.
14. Query the top 5 products based on sales.
15. Two months sales queries.



MySQL Learnings

Working on this 15-task SQL assignment for balleras_db was a real hands-on learning experience for me. Doing everything in MySQL inside XAMPP helped me actually see how databases work behind the scenes instead of just reading about them in theory. Under the guidance of Sir Christian G. Igay, I went from just trying to get basic queries to run without syntax errors to actually understanding how to pull useful information out of different tables.

One of the biggest eye-openers for me during this activity was finally grasping how JOIN statements work. At first, having our data split across three separate tables, data_customers, data_products, and data_sales, felt a bit confusing to track. But once I realized that a JOIN basically bridges those tables together to build a single temporary virtual table on the fly, it all clicked. It showed me how relational databases keep raw data clean and organized without messing up the original records when we want to view them together.

As the tasks progressed, I got to practice combining different SQL functions to make sense of the sales numbers. Using CONCAT() was really helpful for combining first and last names so the output looked neat and easy to read. The real challenge came with aggregate functions like SUM() and AVG(), especially when pairing them with GROUP BY. Learning to think of GROUP BY as a "clustering machine" helped me see how SQL bundles multiple individual receipts into a single line to calculate things like a customer's total spending or a product's average sale price.

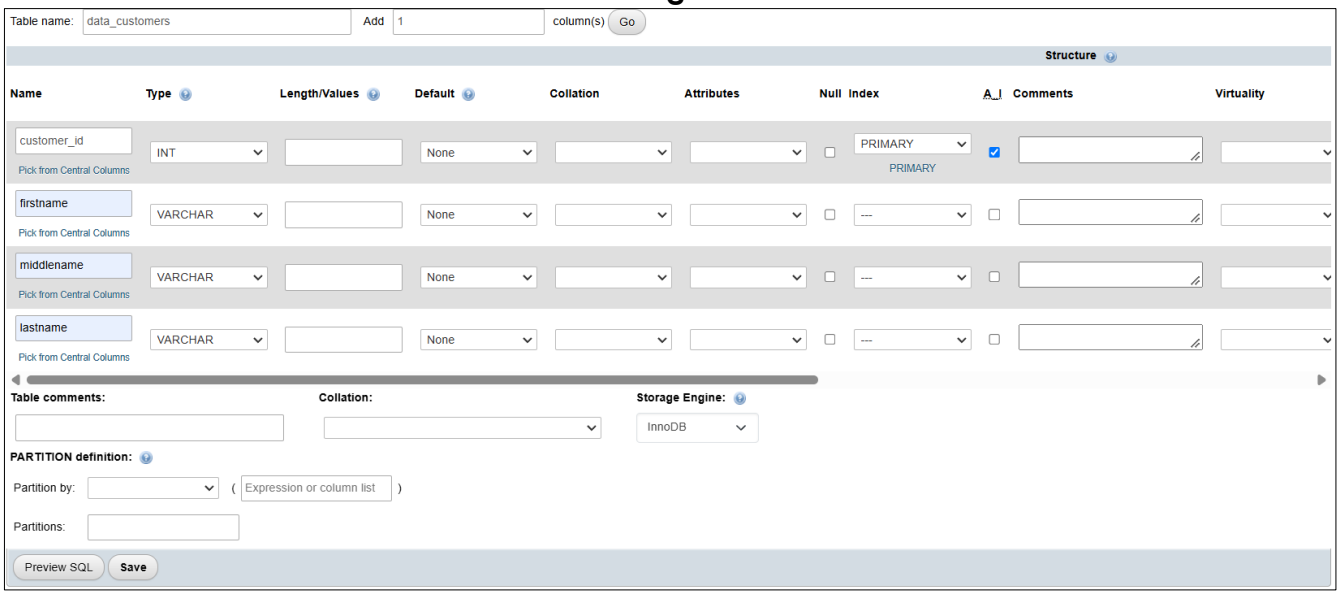
Towards the end of the assignment, tasks like finding the top spenders, ranking best-selling products, and filtering sales across a specific two-month range showed me the bigger picture of data analytics. It made me realize that writing code isn't just a technical exercise, it's about answering actual business questions. Knowing which item generates the most revenue or tracking how sales shift over time are the exact kinds of insights that help businesses make smart decisions.

Overall, completing this 15-task activity built a solid foundation for how I approach databases. Breaking down each query line by line, from the SELECT list all the way to ORDER BY and LIMIT, gave me the confidence to not just run the code, but to actually explain and defend how every part works. I feel much more comfortable using MySQL now and look forward to applying these skills to future data analytics projects.

Vhon Xander R. Balleras
Trainee

TABLES

Creating Table:



Sales Table:

	sale_id	customer_id	product_id	quantity	total_price	sale_date
Edit Copy Delete	1	1	1	2	900.00	2026-10-01
Edit Copy Delete	2	1	3	1	1500.00	2026-10-02
Edit Copy Delete	3	2	4	5	225.00	2026-10-03
Edit Copy Delete	4	3	11	1	850.00	2026-10-04
Edit Copy Delete	5	4	2	3	660.00	2026-10-05
Edit Copy Delete	6	5	6	2	700.00	2026-10-06
Edit Copy Delete	7	6	12	1	1250.00	2026-10-07
Edit Copy Delete	8	7	1	1	450.00	2026-10-08
Edit Copy Delete	9	8	7	2	560.00	2026-10-09
Edit Copy Delete	10	9	23	1	499.00	2026-10-10
Edit Copy Delete	11	10	24	2	700.00	2026-10-11
Edit Copy Delete	12	11	25	3	840.00	2026-10-12
Edit Copy Delete	13	12	1	1	450.00	2026-10-13
Edit Copy Delete	14	13	3	2	3000.00	2026-10-14
Edit Copy Delete	15	14	5	4	720.00	2026-10-15
Edit Copy Delete	16	15	11	1	850.00	2026-10-16
Edit Copy Delete	17	1	2	2	440.00	2026-10-17
Edit Copy Delete	18	2	3	1	1500.00	2026-10-18
Edit Copy Delete	19	3	1	3	1350.00	2026-10-19
Edit Copy Delete	20	4	12	1	1250.00	2026-10-20
Edit Copy Delete	21	5	4	10	450.00	2026-10-21
Edit Copy Delete	22	6	6	1	350.00	2026-10-22
Edit Copy Delete	23	7	25	2	560.00	2026-10-23
Edit Copy Delete	24	8	23	1	499.00	2026-10-24
Edit Copy Delete	25	9	2	4	880.00	2026-10-25
Edit Copy Delete	26	10	1	2	900.00	2026-10-26
Edit Copy Delete	27	11	3	1	1500.00	2026-10-27
Edit Copy Delete	28	12	11	1	850.00	2026-10-28
Edit Copy Delete	29	13	12	1	1250.00	2026-10-29
Edit Copy Delete	30	14	1	1	450.00	2026-10-30

Customers Table:

	customer_id	first_name	last_name	email	phone	city
Edit Copy Delete	1	Juan	Dela Cruz	juan.delacruz@email.com	09123456789	Manila
Edit Copy Delete	2	Maria	Santos	maria.santos@email.com	09234567890	Cebu City
Edit Copy Delete	3	Jose	Rizal	jose.rizal@email.com	09345678901	Calcalog
Edit Copy Delete	4	Ana	Reyes	ana.reyes@email.com	09456789012	Davao City
Edit Copy Delete	5	Pedro	Penduko	pedro.penduko@email.com	09567890123	Quezon City
Edit Copy Delete	6	Clara	Lim	clara.lim@email.com	09678901234	Makati
Edit Copy Delete	7	Antonio	Luna	antonio.luna@email.com	09789012345	Iloilo City
Edit Copy Delete	8	Gabriela	Silang	gabriela.silang@email.com	09890123456	Vigan
Edit Copy Delete	9	Emilio	Aguinaldo	emilio.aguinaldo@email.com	09901234567	Kawit
Edit Copy Delete	10	Andres	Bonifacio	andres.bonifacio@email.com	09012345678	Manila
Edit Copy Delete	11	Melchora	Aquino	melchora.aquino@email.com	09112233445	Quezon City
Edit Copy Delete	12	Tandang	Sora	tandang.sora@email.com	09223344556	Caloocan
Edit Copy Delete	13	Ramon	Magsaysay	ramon.magsaysay@email.com	09334455667	Zambales
Edit Copy Delete	14	Manuel	Quezon	manuel.quezon@email.com	09445566778	Tayabas
Edit Copy Delete	15	Sergio	Osmena	sergio.osmena@email.com	09556677889	Cebu
Edit Copy Delete	16	Elpidio	Quirino	elpidio.quirino@email.com	09667788990	Vigan
Edit Copy Delete	17	Carlos	Garcia	carlos.garcia@email.com	09778899001	Bohol
Edit Copy Delete	18	Diosdado	Macapagal	diosdado.macapagal@email.com	09889900112	Pampanga
Edit Copy Delete	19	Ferdinand	Marcos	ferdinand.marcos@email.com	09990011223	Ilocos Norte
Edit Copy Delete	20	Corazon	Aquino	corazon.aquino@email.com	09101112233	Tarlac
Edit Copy Delete	21	Fidel	Ramos	fidel.ramos@email.com	09212233445	Lingayen
Edit Copy Delete	22	Joseph	Estrada	joseph.estrada@email.com	09323344556	Manila
Edit Copy Delete	23	Gloria	Arroyo	gloria.arroyo@email.com	09434455667	Pampanga
Edit Copy Delete	24	Benigno	Aquino	benigno.aquino@email.com	09545566778	Tarlac
Edit Copy Delete	25	Rodrigo	Duterte	rodrigo.duterte@email.com	09656677889	Davao City
Edit Copy Delete	26	Bongbong	Marcos	bongbong.marcos@email.com	09767788990	Ilocos Norte
Edit Copy Delete	27	Sara	Duterte	sara.duterte@email.com	09878899001	Davao City
Edit Copy Delete	28	Isko	Moreno	isko.moreno@email.com	09989900112	Manila
Edit Copy Delete	29	Leni	Robredo	leni.robredo@email.com	09192939495	Naga
Edit Copy Delete	30	Kiko	Pangilinan	kiko.pangilinan@email.com	09293949506	Manila

Products Table:

	product_id	product_name	category	price	stock_quantity
Edit Copy Delete	1	Sketch Pencil Set (35-Piece)	Art Supplies	450.00	120
Edit Copy Delete	2	A4 Bond Paper (500 Sheets)	Office Supplies	220.00	200
Edit Copy Delete	3	Canva Pro Subscription Card	Digital	1500.00	50
Edit Copy Delete	4	Spiral Notebook	School Supplies	45.00	300
Edit Copy Delete	5	Gel Pen Black (Box of 12)	School Supplies	180.00	150
Edit Copy Delete	6	Color Marker Set (24 Colors)	Art Supplies	350.00	80
Edit Copy Delete	7	Watercolor Paint Pan Set	Art Supplies	280.00	90
Edit Copy Delete	8	Sticky Notes (Pack of 4)	Office Supplies	75.00	250
Edit Copy Delete	9	Eraser (Dust-Free)	School Supplies	15.00	500
Edit Copy Delete	10	Plastic Ruler 12-inch	School Supplies	20.00	400
Edit Copy Delete	11	Scientific Calculator	Electronics	850.00	60
Edit Copy Delete	12	LCD Screen Replacement 6.78"	Electronics	1250.00	25
Edit Copy Delete	13	Storage Organizer Box	Home & Living	320.00	70
Edit Copy Delete	14	Thermal Laminating Pouch (100pcs)	Office Supplies	250.00	110
Edit Copy Delete	15	Permanent Marker (Black)	Office Supplies	35.00	220
Edit Copy Delete	16	Correction Tape	School Supplies	40.00	180
Edit Copy Delete	17	Scissors (Stainless Steel)	Office Supplies	60.00	130
Edit Copy Delete	18	Glue Stick (Large)	School Supplies	25.00	210
Edit Copy Delete	19	Stapler Heavy Duty	Office Supplies	190.00	45
Edit Copy Delete	20	Staple Wire (Box)	Office Supplies	30.00	300
Edit Copy Delete	21	File Folder (Long, 10pcs)	Office Supplies	90.00	140
Edit Copy Delete	22	Whiteboard Marker Set	Office Supplies	120.00	95
Edit Copy Delete	23	Desk LED Lamp	Electronics	499.00	40
Edit Copy Delete	24	Wireless Mouse	Electronics	350.00	85
Edit Copy Delete	25	USB Flash Drive 32GB	Electronics	280.00	100
Edit Copy Delete	26	Notebook Filler Paper	School Supplies	65.00	160
Edit Copy Delete	27	Craft Paper Pack (50 sheets)	Art Supplies	110.00	75
Edit Copy Delete	28	Double-Sided Tape	Office Supplies	45.00	190
Edit Copy Delete	29	Binder Clips (Assorted Sizes)	Office Supplies	55.00	230
Edit Copy Delete	30	Drawing Compass Set	School Supplies	130.00	65

TASK 1: Add product, customer, sales.

Products:

```
1 INSERT INTO data_products (product_name, category, price, stock_quantity)
2 VALUES ('Bluetooth Portable Speaker', 'Electronics', 1200.00, 50);
```

1 row inserted.
Inserted row id: 31 (Query took 0.0023 seconds.)

```
INSERT INTO data_products (product_name, category, price, stock_quantity) VALUES ('Bluetooth Portable Speaker', 'Electronics', 1200.00, 50);
```

☐ Edit Copy Delete	31	Bluetooth Portable Speaker	Electronics	1200.00	50	

Customers:

```
1 INSERT INTO data_customer (first_name, last_name, email, phone, city)
2 VALUES ('Sarah', 'Geronimo', 'sarah.geronimo@email.com', '09198765432', 'Manila');
```

1 row inserted.
Inserted row id: 31 (Query took 0.0020 seconds.)

```
INSERT INTO data_customer (first_name, last_name, email, phone, city) VALUES ('Sarah', 'Geronimo', 'sarah.geronimo@email.com', '09198765432', 'Manila');
```

☐ Edit Copy Delete	31	Sarah	Geronimo	sarah.geronimo@email.com	09198765432	Manila

Sales:

```
1 INSERT INTO data_sales (customer_id, product_id, quantity, total_price, sale_date)
2 VALUES ((SELECT customer_id FROM data_customers WHERE email = 'sarah.geronimo@email.com' LIMIT 1), (SELECT product_id FROM data_products WHERE
  product_name = 'Bluetooth Portable Speaker' LIMIT 1), 2, 2400.00, '2026-10-30');
```

1 row inserted.
Inserted row id: 31 (Query took 0.0034 seconds.)

```
INSERT INTO data_sales (customer_id, product_id, quantity, total_price, sale_date) VALUES ((SELECT customer_id FROM data_customers WHERE email = 'sarah.geronimo@email.com' LIMIT 1), (SELECT product_id
FROM data_products WHERE product_name = 'Bluetooth Portable Speaker' LIMIT 1), 2, 2400.00, '2026-10-30');
```

☐ Edit Copy Delete	31	31	31	2	2400.00	2026-10-30

Since, we already created the costumer and the product's data in their table, we can just reference them to get their information.

TASK 2: Update product and customer.

Product:

Here's the data before.

☐	Edit Copy Delete	31	Bluetooth Portable Speaker	Electronics	1200.00	50

In this case, I want to change the Bluetooth Portable Speaker's Price and Stock Quantity.

```
1 UPDATE data_products
2 SET price = 1300.00, stock_quantity = 45
3 WHERE product_name = 'Bluetooth Portable Speaker';
```

✔ 1 row affected. (Query took 0.0016 seconds.)

```
UPDATE data_products SET price = 1300.00, stock_quantity = 45 WHERE product_name = 'Bluetooth Portable Speaker';
```

☐	Edit Copy Delete	31	Bluetooth Portable Speaker	Electronics	1300.00	45

Result: Bluetooth Portable Speaker's Price from 1200.00 to 1300.00, and Stock Quantity from 50 > 45.

Customer:

Here's the data before.

☐	Edit Copy Delete	31	Sarah	Geronimo	sarah.geronimo@email.com	09198765432	Manila

In this case, I want to change Sarah Geronimo's City and Phone information.

```
1 UPDATE data_customers
2 SET city = 'Cebu City', phone = '09998887766'
3 WHERE first_name = 'Sarah' AND last_name = 'Geronimo';
```

✔ 1 row affected. (Query took 0.0017 seconds.)

```
UPDATE data_customers SET city = 'Cebu City', phone = '09998887766' WHERE first_name = 'Sarah' AND last_name = 'Geronimo';
```

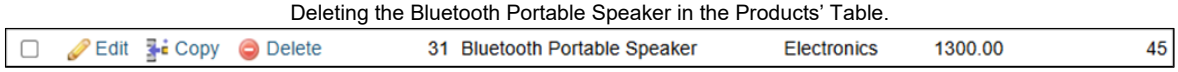
☐	Edit Copy Delete	31	Sarah	Geronimo	sarah.geronimo@email.com	09998887766	Cebu City

Result: Sarah Geronimo's City from Manila > Cebu City, and Phone from 09198765432 > 09998887766.

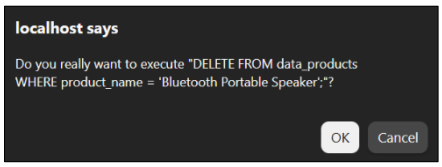
TASK 3: Delete product and customer.

Product:

Deleting the Bluetooth Portable Speaker in the Products' Table.




```
1 DELETE FROM data_products
2 WHERE product_name = 'Bluetooth Portable Speaker';
```

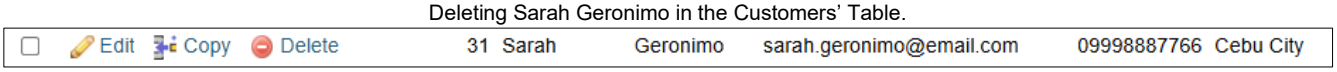


1 row deleted. (Query took 0.0018 seconds.)

```
DELETE FROM data_products WHERE product_name = 'Bluetooth Portable Speaker';
```

Customer:

Deleting Sarah Geronimo in the Customers' Table.




```
1 DELETE FROM data_customers
2 WHERE first_name = 'Sarah' AND last_name = 'Geronimo';
```



1 row deleted. (Query took 0.0025 seconds.)

```
DELETE FROM data_customers WHERE first_name = 'Sarah' AND last_name = 'Geronimo';
```

NOTE:

Deleting Sarah Geronimo and the Bluetooth Portable Speaker will also automatically delete the Sales that is referenced/related to them.

TASK 4: Query customers.

All Query:

```
1 SELECT * FROM data_customers;
```

Showing rows 0 - 29 (30 total, Query took 0.0004 seconds.)

```
SELECT * FROM data_customers;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 | Filter rows: Search this table | Sort by key: None

Extra options

	customer_id	first_name	last_name	email	phone	city
Edit Copy Delete	1	Juan	Dela Cruz	juan.delacruz@email.com	09123456789	Manila
Edit Copy Delete	2	Maria	Santos	maria.santos@email.com	09234567890	Cebu City
Edit Copy Delete	3	Jose	Rizal	jose.rizal@email.com	09345678901	Calcalog
Edit Copy Delete	4	Ana	Reyes	ana.reyes@email.com	09456789012	Davao City
Edit Copy Delete	5	Pedro	Penduko	pedro.penduko@email.com	09567890123	Quezon City
Edit Copy Delete	6	Clara	Lim	clara.lim@email.com	09678901234	Makati
Edit Copy Delete	7	Antonio	Luna	antonio.luna@email.com	09789012345	Iloilo City
Edit Copy Delete	8	Gabriela	Silang	gabriela.silang@email.com	09890123456	Vigan
Edit Copy Delete	9	Emilio	Aguinaldo	emilio.aguinaldo@email.com	09901234567	Kawit
Edit Copy Delete	10	Andres	Bonifacio	andres.bonifacio@email.com	09012345678	Manila
Edit Copy Delete	11	Melchora	Aquino	melchora.aquino@email.com	09112233445	Quezon City
Edit Copy Delete	12	Tandang	Sora	tandang.sora@email.com	09223344556	Caloocan
Edit Copy Delete	13	Ramon	Magsaysay	ramon.magsaysay@email.com	09334455667	Zambales
Edit Copy Delete	14	Manuel	Quezon	manuel.quezon@email.com	09445566778	Tayabas
Edit Copy Delete	15	Sergio	Osmena	sergio.osmena@email.com	09556677889	Cebu
Edit Copy Delete	16	Elpidio	Quirino	elpidio.quirino@email.com	09667788990	Vigan
Edit Copy Delete	17	Carlos	Garcia	carlos.garcia@email.com	09778899001	Bohol
Edit Copy Delete	18	Diosdado	Macapagal	diosdado.macapagal@email.com	09889900112	Pampanga
Edit Copy Delete	19	Ferdinand	Marcos	ferdinand.marcos@email.com	09990011223	Ilocos Norte
Edit Copy Delete	20	Corazon	Aquino	corazon.aquino@email.com	09101112233	Tarlac
Edit Copy Delete	21	Fidel	Ramos	fidel.ramos@email.com	09212233445	Lingayen
Edit Copy Delete	22	Joseph	Estrada	joseph.estrada@email.com	09323344556	Manila
Edit Copy Delete	23	Gloria	Arroyo	gloria.arroyo@email.com	09434455667	Pampanga
Edit Copy Delete	24	Benigno	Aquino	benigno.aquino@email.com	09545566778	Tarlac
Edit Copy Delete	25	Rodrigo	Duterte	rodrigo.duterte@email.com	09656677889	Davao City
Edit Copy Delete	26	Bongbong	Marcos	bongbong.marcos@email.com	09767788990	Ilocos Norte
Edit Copy Delete	27	Sara	Duterte	sara.duterte@email.com	09878899001	Davao City
Edit Copy Delete	28	Isko	Moreno	isko.moreno@email.com	09989900112	Manila
Edit Copy Delete	29	Leni	Robredo	leni.robredo@email.com	09102939495	Naga
Edit Copy Delete	30	Kiko	Pangilinan	kiko.pangilinan@email.com	09293949506	Manila

Specific Query:

```
1 SELECT * FROM data_customers
2 WHERE first_name = 'Juan' AND last_name = 'Dela Cruz';
```

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT * FROM data_customers WHERE first_name = 'Juan' AND last_name = 'Dela Cruz';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	customer_id	first_name	last_name	email	phone	city
Edit Copy Delete	1	Juan	Dela Cruz	juan.delacruz@email.com	09123456789	Manila

TASK 5: Query sales.

All Query:

```
1 SELECT * FROM data_sales;
```

Showing rows 0 - 29 (30 total, Query took 0.0004 seconds.)

```
SELECT * FROM `data_sales`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 Filter rows: Search this table Sort by key: None

Extra options

	sale_id	customer_id	product_id	quantity	total_price	sale_date
Edit Copy Delete	1	1	1	2	900.00	2026-10-01
Edit Copy Delete	2	1	3	1	1500.00	2026-10-02
Edit Copy Delete	3	2	4	5	225.00	2026-10-03
Edit Copy Delete	4	3	11	1	850.00	2026-10-04
Edit Copy Delete	5	4	2	3	660.00	2026-10-05
Edit Copy Delete	6	5	6	2	700.00	2026-10-06
Edit Copy Delete	7	6	12	1	1250.00	2026-10-07
Edit Copy Delete	8	7	1	1	450.00	2026-10-08
Edit Copy Delete	9	8	7	2	560.00	2026-10-09
Edit Copy Delete	10	9	23	1	499.00	2026-10-10
Edit Copy Delete	11	10	24	2	700.00	2026-10-11
Edit Copy Delete	12	11	25	3	840.00	2026-10-12
Edit Copy Delete	13	12	1	1	450.00	2026-10-13
Edit Copy Delete	14	13	3	2	3000.00	2026-10-14
Edit Copy Delete	15	14	5	4	720.00	2026-10-15
Edit Copy Delete	16	15	11	1	850.00	2026-10-16
Edit Copy Delete	17	1	2	2	440.00	2026-10-17
Edit Copy Delete	18	2	3	1	1500.00	2026-10-18
Edit Copy Delete	19	3	1	3	1350.00	2026-10-19
Edit Copy Delete	20	4	12	1	1250.00	2026-10-20
Edit Copy Delete	21	5	4	10	450.00	2026-10-21
Edit Copy Delete	22	6	6	1	350.00	2026-10-22
Edit Copy Delete	23	7	25	2	560.00	2026-10-23
Edit Copy Delete	24	8	23	1	499.00	2026-10-24
Edit Copy Delete	25	9	2	4	880.00	2026-10-25
Edit Copy Delete	26	10	1	2	900.00	2026-10-26
Edit Copy Delete	27	11	3	1	1500.00	2026-10-27
Edit Copy Delete	28	12	11	1	850.00	2026-10-28
Edit Copy Delete	29	13	12	1	1250.00	2026-10-29
Edit Copy Delete	30	14	1	1	450.00	2026-10-30

Specific Query:

```
1 SELECT * FROM data_sales
2 WHERE sale_id = 1;
```

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT * FROM data_sales WHERE sale_id = 1;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

	sale_id	customer_id	product_id	quantity	total_price	sale_date
Edit Copy Delete	1	1	1	2	900.00	2026-10-01

TASK 6: Query products.

All Query:

```
1 SELECT * FROM data_products;
```

Showing rows 0 - 29 (30 total, Query took 0.0004 seconds.)

```
SELECT * FROM data_products;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 Filter rows: Search this table Sort by key: None

Extra options

	product_id	product_name	category	price	stock_quantity
Edit Copy Delete	1	Sketch Pencil Set (35-Piece)	Art Supplies	450.00	120
Edit Copy Delete	2	A4 Bond Paper (500 Sheets)	Office Supplies	220.00	200
Edit Copy Delete	3	Canva Pro Subscription Card	Digital	1500.00	50
Edit Copy Delete	4	Spiral Notebook	School Supplies	45.00	300
Edit Copy Delete	5	Gel Pen Black (Box of 12)	School Supplies	180.00	150
Edit Copy Delete	6	Color Marker Set (24 Colors)	Art Supplies	350.00	80
Edit Copy Delete	7	Watercolor Paint Pan Set	Art Supplies	280.00	90
Edit Copy Delete	8	Sticky Notes (Pack of 4)	Office Supplies	75.00	250
Edit Copy Delete	9	Eraser (Dust-Free)	School Supplies	15.00	500
Edit Copy Delete	10	Plastic Ruler 12-inch	School Supplies	20.00	400
Edit Copy Delete	11	Scientific Calculator	Electronics	850.00	60
Edit Copy Delete	12	LCD Screen Replacement 6.78"	Electronics	1250.00	25
Edit Copy Delete	13	Storage Organizer Box	Home & Living	320.00	70
Edit Copy Delete	14	Thermal Laminating Pouch (100pcs)	Office Supplies	250.00	110
Edit Copy Delete	15	Permanent Marker (Black)	Office Supplies	35.00	220
Edit Copy Delete	16	Correction Tape	School Supplies	40.00	180
Edit Copy Delete	17	Scissors (Stainless Steel)	Office Supplies	60.00	130
Edit Copy Delete	18	Glue Stick (Large)	School Supplies	25.00	210
Edit Copy Delete	19	Stapler Heavy Duty	Office Supplies	190.00	45
Edit Copy Delete	20	Staple Wire (Box)	Office Supplies	30.00	300
Edit Copy Delete	21	File Folder (Long, 10pcs)	Office Supplies	90.00	140
Edit Copy Delete	22	Whiteboard Marker Set	Office Supplies	120.00	95
Edit Copy Delete	23	Desk LED Lamp	Electronics	499.00	40
Edit Copy Delete	24	Wireless Mouse	Electronics	350.00	85
Edit Copy Delete	25	USB Flash Drive 32GB	Electronics	280.00	100
Edit Copy Delete	26	Notebook Filler Paper	School Supplies	65.00	160
Edit Copy Delete	27	Craft Paper Pack (50 sheets)	Art Supplies	110.00	75
Edit Copy Delete	28	Double-Sided Tape	Office Supplies	45.00	190
Edit Copy Delete	29	Binder Clips (Assorted Sizes)	Office Supplies	55.00	230
Edit Copy Delete	30	Drawing Compass Set	School Supplies	130.00	65

Specific Query:

```
1 SELECT * FROM data_products
2 WHERE product_name = 'Sketch Pencil Set (35-Piece)';
```

Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT * FROM data_products WHERE product_name = 'Sketch Pencil Set (35-Piece)';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

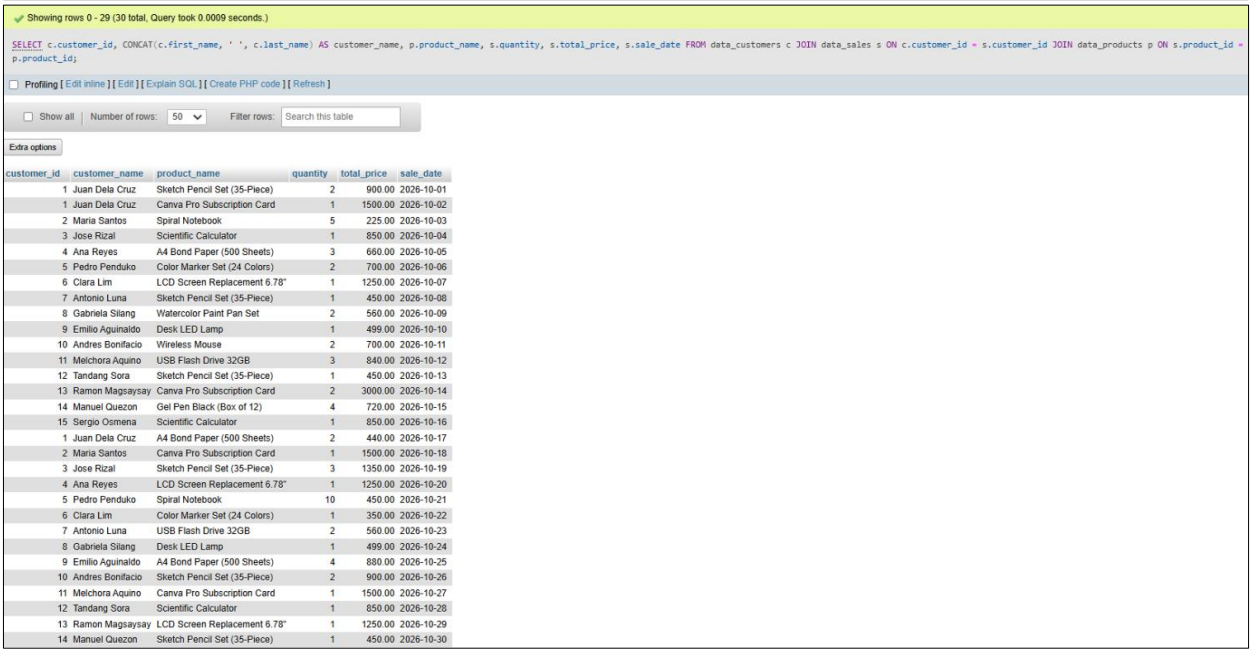
Extra options

	product_id	product_name	category	price	stock_quantity
Edit Copy Delete	1	Sketch Pencil Set (35-Piece)	Art Supplies	450.00	120

TASK 7: Query customers purchases.

All Query

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    p.product_name,
    s.quantity,
    s.total_price,
    s.sale_date
FROM data_customers c
JOIN data_sales s ON c.customer_id = s.customer_id
JOIN data_products p ON s.product_id = p.product_id;
```



Showing rows 0 - 29 (30 total, Query took 0.0009 seconds.)

```
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, p.product_name, s.quantity, s.total_price, s.sale_date FROM data_customers c JOIN data_sales s ON c.customer_id = s.customer_id JOIN data_products p ON s.product_id = p.product_id;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 | Filter rows: Search this table

Extra options

customer_id	customer_name	product_name	quantity	total_price	sale_date
1	Juan Dela Cruz	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-01
1	Juan Dela Cruz	Canva Pro Subscription Card	1	1500.00	2026-10-02
2	Maria Santos	Spiral Notebook	5	225.00	2026-10-03
3	Jose Rizal	Scientific Calculator	1	850.00	2026-10-04
4	Ana Reyes	A4 Bond Paper (500 Sheets)	3	660.00	2026-10-05
5	Pedro Penduko	Color Marker Set (24 Colors)	2	700.00	2026-10-06
6	Clara Lim	LCD Screen Replacement 6.78"	1	1250.00	2026-10-07
7	Antonio Luna	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-08
8	Gabriela Silang	Watercolor Paint Pan Set	2	560.00	2026-10-09
9	Emilio Aguinaldo	Desk LED Lamp	1	499.00	2026-10-10
10	Andres Bonifacio	Wireless Mouse	2	700.00	2026-10-11
11	Melchora Aquino	USB Flash Drive 32GB	3	840.00	2026-10-12
12	Tandang Sora	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-13
13	Ramon Magsaysay	Canva Pro Subscription Card	2	3000.00	2026-10-14
14	Manuel Quezon	Gel Pen Black (Box of 12)	4	720.00	2026-10-15
15	Sergio Osmena	Scientific Calculator	1	850.00	2026-10-16
1	Juan Dela Cruz	A4 Bond Paper (500 Sheets)	2	440.00	2026-10-17
2	Maria Santos	Canva Pro Subscription Card	1	1500.00	2026-10-18
3	Jose Rizal	Sketch Pencil Set (35-Piece)	3	1350.00	2026-10-19
4	Ana Reyes	LCD Screen Replacement 6.78"	1	1250.00	2026-10-20
5	Pedro Penduko	Spiral Notebook	10	450.00	2026-10-21
6	Clara Lim	Color Marker Set (24 Colors)	1	350.00	2026-10-22
7	Antonio Luna	USB Flash Drive 32GB	2	560.00	2026-10-23
8	Gabriela Silang	Desk LED Lamp	1	499.00	2026-10-24
9	Emilio Aguinaldo	A4 Bond Paper (500 Sheets)	4	880.00	2026-10-25
10	Andres Bonifacio	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-26
11	Melchora Aquino	Canva Pro Subscription Card	1	1500.00	2026-10-27
12	Tandang Sora	Scientific Calculator	1	850.00	2026-10-28
13	Ramon Magsaysay	LCD Screen Replacement 6.78"	1	1250.00	2026-10-29
14	Manuel Quezon	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-30

To see what products the customers have bought, I used a JOIN query to connect the data_costumers table with the data_sales and data_products tables.

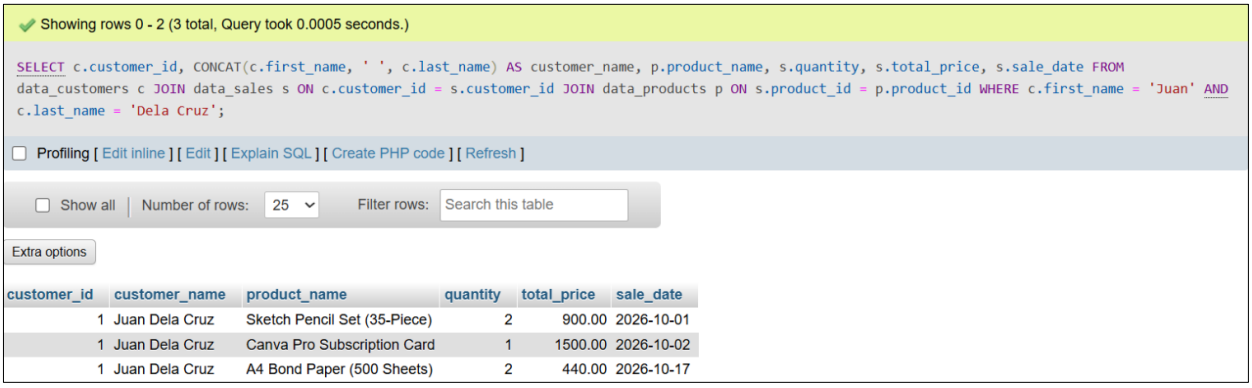
SELECT -- *Basically asking to show these following.*
  c.customer_id, -- *Creates a column in the virtual table for the ID of the customers.*
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- *CONCAT puts the first names and last names together into one column with a space in between.*
  p.product_name, -- *Creates a column in the virtual table for the name of the products that the customers bought.*
  s.quantity, -- *Creates a column in the virtual table for the quantity bought.*
  s.total_price, -- *Creates a column in the virtual table for the total price of the purchases.*
  s.sale_date -- *Creates a column in the virtual table for the date of the purchases.*

FROM data_customers c -- *Sets "c" as the table alias for "data_customers".*
JOIN data_sales s ON c.customer_id = s.customer_id -- *Sets "s" as the table alias for "data_sales", and bridges/matches the customer IDs from customers and sales tables.*
JOIN data_products p ON s.product_id = p.product_id -- *Sets "p" as the table alias for "data_products", and bridges/matches the product IDs from sales and products tables.*

TASK 7: Query customers purchases.

Specific Query

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    p.product_name,
    s.quantity,
    s.total_price,
    s.sale_date
FROM data_costumers c
JOIN data_sales s ON c.customer_id = s.customer_id
JOIN data_products p ON s.product_id = p.product_id
WHERE c.first_name = 'Juan' AND c.last_name = 'Dela Cruz';
```



Showing rows 0 - 2 (3 total, Query took 0.0005 seconds.)

```
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, p.product_name, s.quantity, s.total_price, s.sale_date FROM
data_customers c JOIN data_sales s ON c.customer_id = s.customer_id JOIN data_products p ON s.product_id = p.product_id WHERE c.first_name = 'Juan' AND
c.last_name = 'Dela Cruz';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

customer_id	customer_name	product_name	quantity	total_price	sale_date
1	Juan Dela Cruz	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-01
1	Juan Dela Cruz	Canva Pro Subscription Card	1	1500.00	2026-10-02
1	Juan Dela Cruz	A4 Bond Paper (500 Sheets)	2	440.00	2026-10-17

To see what products the customers have bought, I used a JOIN query to connect the data_costumers table with the data_sales and data_products tables.

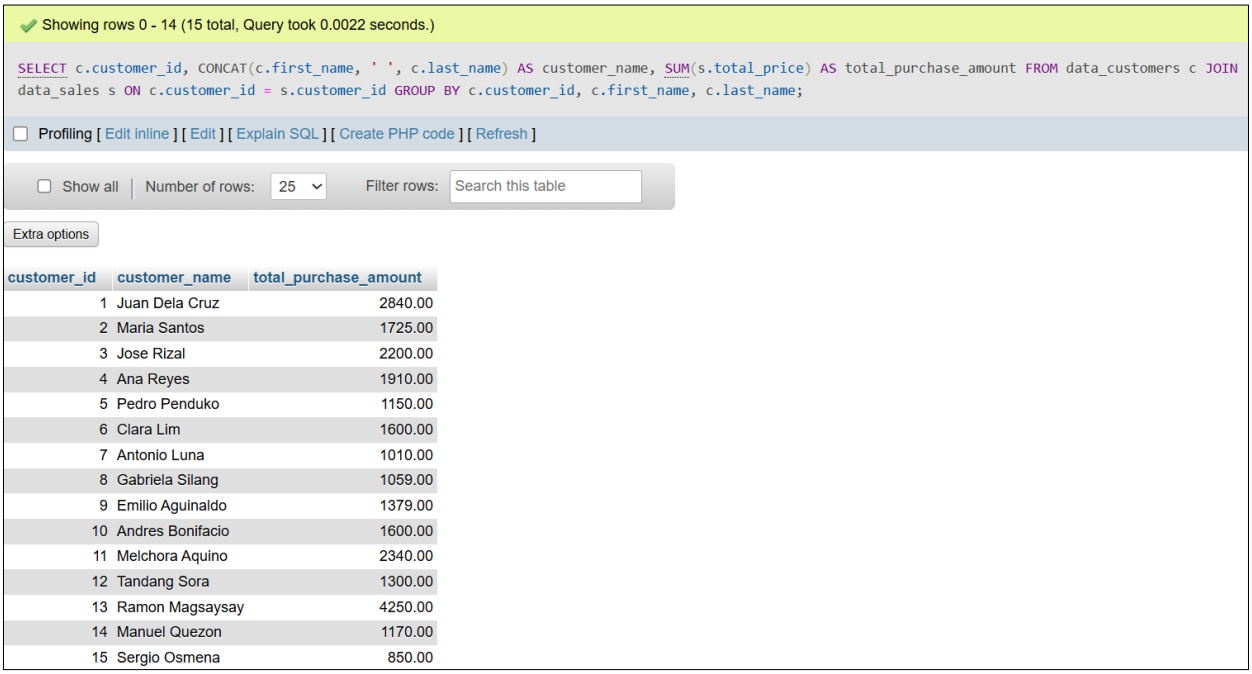
SELECT -- *Basically asking to show these following.*
  c.customer_id, -- *Creates a column in the virtual table for the ID of the customer.*
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- *CONCAT puts the first name and last name together into one column with a space in between.*
  p.product_name, -- *Creates a column in the virtual table for the name of the products that the customer bought.*
  s.quantity, -- *Creates a column in the virtual table for the quantity bought.*
  s.total_price, -- *Creates a column in the virtual table for the total price of the purchases.*
  s.sale_date -- *Creates a column in the virtual table for the date of the purchases.*

FROM data_customers c -- *Sets "c" as the table alias for "data_customers".*
JOIN data_sales s ON c.customer_id = s.customer_id -- *Sets "s" as the table alias for "data_sales", and bridges/matches the customer ID from customers and sales tables.*
JOIN data_products p ON s.product_id = p.product_id -- *Sets "p" as the table alias for "data_products", and bridges/matches the product IDs from sales and products tables.*
WHERE c.first_name = 'Juan' AND c.last_name = 'Dela Cruz'; -- *Filters the virtual table to only show records for this specific customer.*

TASK 8: Query customers total purchase amount.

All Query

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    SUM(s.total_price) AS total_purchase_amount
FROM data_customers c
JOIN data_sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```



Showing rows 0 - 14 (15 total, Query took 0.0022 seconds.)

```
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, SUM(s.total_price) AS total_purchase_amount FROM data_customers c JOIN data_sales s ON c.customer_id = s.customer_id GROUP BY c.customer_id, c.first_name, c.last_name;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

customer_id	customer_name	total_purchase_amount
1	Juan Dela Cruz	2840.00
2	Maria Santos	1725.00
3	Jose Rizal	2200.00
4	Ana Reyes	1910.00
5	Pedro Penduko	1150.00
6	Clara Lim	1600.00
7	Antonio Luna	1010.00
8	Gabriela Silang	1059.00
9	Emilio Aguinaldo	1379.00
10	Andres Bonifacio	1600.00
11	Melchora Aquino	2340.00
12	Tandang Sora	1300.00
13	Ramon Magsaysay	4250.00
14	Manuel Quezon	1170.00
15	Sergio Osmena	850.00

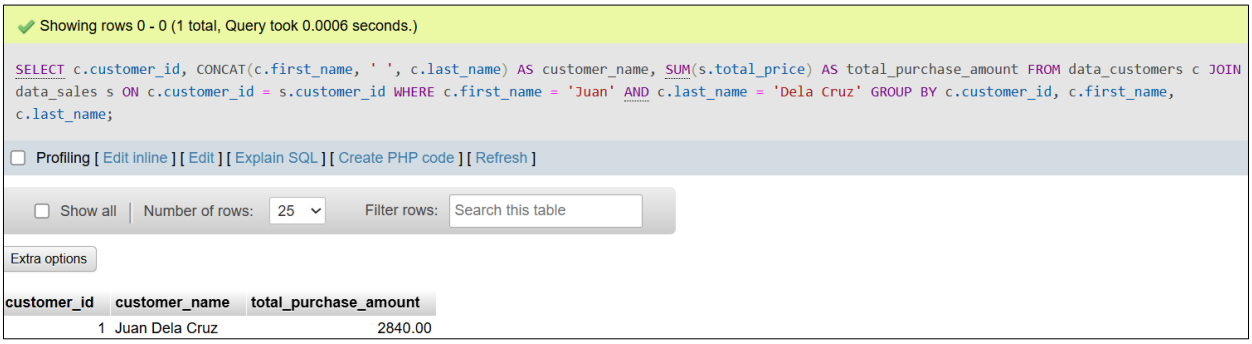
Since I want to add up how much each customer has spent in total, I used two new tools:
SUM() (to add the numbers together) and GROUP BY (to group the purchases by each individual person).

```
SELECT -- Basically asking to show these following.
   c.customer_id, -- Creates a column in the virtual table for the ID of the customers.
   CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- CONCAT puts the first names and last names together into one column
                                                               with a space in between.
   SUM(s.total_price) AS total_purchase_amount -- SUM adds up all of the customers' purchase amounts into one grand total.
FROM data_customers c -- Sets "c" as the table alias for "data_customers".
JOIN data_sales s ON c.customer_id = s.customer_id -- Bridges/matches the customer IDs from the customers and sales tables.
GROUP BY c.customer_id, c.first_name, c.last_name; -- Gathers all of customers' separate purchase receipts and packs them into a single
                                                      row so the SUM() function can calculate their overall total.
```

TASK 8: Query customers total purchase amount.

Specific Query

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    SUM(s.total_price) AS total_purchase_amount
FROM data_customers c
JOIN data_sales s ON c.customer_id = s.customer_id
WHERE c.first_name = 'Juan' AND c.last_name = 'Dela Cruz'
GROUP BY c.customer_id, c.first_name, c.last_name;
```


Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)

SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, SUM(s.total_price) AS total_purchase_amount FROM data_customers c JOIN data_sales s ON c.customer_id = s.customer_id WHERE c.first_name = 'Juan' AND c.last_name = 'Dela Cruz' GROUP BY c.customer_id, c.first_name, c.last_name;

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

customer_id	customer_name	total_purchase_amount
1	Juan Dela Cruz	2840.00

Since I want to add up how much each customer has spent in total, I used two new tools:
SUM() (to add the numbers together) and GROUP BY (to group the purchases by each individual person).

```
SELECT -- Basically asking to show these following.
   c.customer_id, -- Creates a column in the virtual table for the ID of the customer.
   CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- CONCAT puts the first name and last name together into one column with
                                                               a space in between.
   SUM(s.total_price) AS total_purchase_amount -- SUM adds up all of the customer's purchase amounts into one grand total.
FROM data_customers c -- Sets "c" as the table alias for "data_customers".
JOIN data_sales s ON c.customer_id = s.customer_id -- Bridges/matches the customer ID from the customers and sales tables.
WHERE c.first_name = 'Juan' AND c.last_name = 'Dela Cruz' -- Filters the virtual table to only show records for this specific customer.
GROUP BY c.customer_id, c.first_name, c.last_name; -- Gathers all of Juan's separate purchase receipts and packs them into a single row
                                                      so the SUM() function can calculate his overall total.
```

TASK 9: Query Average sales.

```
1 SELECT
2     AVG(total_price) AS average_sale_amount
3 FROM data_sales;
```

Showing rows 0 - 0 (1 total, Query took 0.0013 seconds.)

```
SELECT AVG(total_price) AS average_sale_amount FROM data_sales;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

average_sale_amount
879.433333

To find the average, I used AVG(), which automatically adds up all the numbers and divides them by how many rows there are.

SELECT -- *Basically asking to show these following.*
AVG(total_price) AS average_sale_amount – *Adds up all the total_price then divide by total number, and renames the resulting column header.*
FROM data_sales; -- *No need for JOIN/bridge since the data is from a single table only.*

TASK 10: Query Top Customers Based on Sales

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    SUM(s.total_price) AS total_spent
FROM data_customers c
JOIN data_sales s ON c.customer_id = s.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spent DESC;
```

Showing rows 0 - 14 (15 total, Query took 0.0006 seconds.)

```sql
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, SUM(s.total_price) AS total_spent FROM data_customers c JOIN data_sales s ON c.customer_id = s.customer_id GROUP BY c.customer_id, c.first_name, c.last_name ORDER BY total_spent DESC;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 Filter rows: Search this table

Extra options

customer_id	customer_name	total_spent
13	Ramon Magsaysay	4250.00
1	Juan Dela Cruz	2840.00
11	Melchora Aquino	2340.00
3	Jose Rizal	2200.00
4	Ana Reyes	1910.00
2	Maria Santos	1725.00
6	Clara Lim	1600.00
10	Andres Bonifacio	1600.00
9	Emilio Aguinaldo	1379.00
12	Tandang Sora	1300.00
14	Manuel Quezon	1170.00
5	Pedro Penduko	1150.00
8	Gabriela Silang	1059.00
7	Antonio Luna	1010.00
15	Sergio Osmena	850.00

To rank customers by who spent the most, I used ORDER BY ... DESC: Sorts the results from highest to lowest total spending.

SELECT -- *Basically asking to show these following.*
c.customer_id, -- *Creates a column in the virtual table for the ID of the customer.*
CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- *CONCAT puts the first name and last name together into one column with a space in between.*
SUM(s.total_price) AS total_spent -- *SUM adds up all of the customer's purchase amounts into one grand total.*
FROM data_customers c -- *Sets "c" as the table alias for "data_customers".*
JOIN data_sales s ON c.customer_id = s.customer_id -- *Sets "c" as the table alias for "data_customers".*
GROUP BY c.customer_id, c.first_name, c.last_name -- *Gathers all of Juan's separate purchase receipts and packs them into a single row so the SUM() function can calculate his overall total.*
ORDER BY total_spent DESC; – *"total_spent" is going to be displayed in descending order.*

TASK 11: Query Product Sales (Specific Product)

```sql
SELECT
    p.product_id,
    p.product_name,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    s.quantity,
    s.total_price,
    s.sale_date
FROM data_sales s
JOIN data_products p ON s.product_id = p.product_id
JOIN data_customers c ON s.customer_id = c.customer_id
WHERE p.product_name = 'Sketch Pencil Set (35-Piece)';
```

Showing rows 0 - 5 (6 total, Query took 0.0005 seconds.)

```sql
SELECT p.product_id, p.product_name, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, s.quantity, s.total_price, s.sale_date FROM data_sales s
JOIN data_products p ON s.product_id = p.product_id JOIN data_customers c ON s.customer_id = c.customer_id WHERE p.product_name = 'Sketch Pencil Set
(35-Piece)';
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

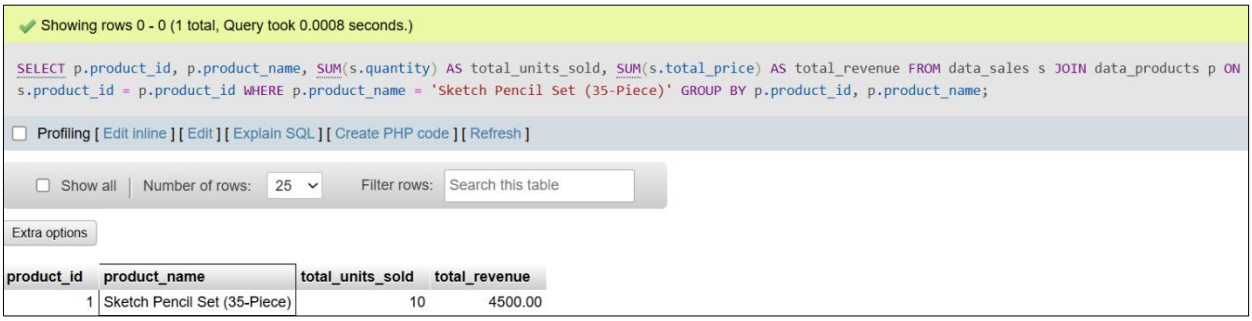
product_id	product_name	customer_name	quantity	total_price	sale_date
1	Sketch Pencil Set (35-Piece)	Juan Dela Cruz	2	900.00	2026-10-01
1	Sketch Pencil Set (35-Piece)	Antonio Luna	1	450.00	2026-10-08
1	Sketch Pencil Set (35-Piece)	Tandang Sora	1	450.00	2026-10-13
1	Sketch Pencil Set (35-Piece)	Jose Rizal	3	1350.00	2026-10-19
1	Sketch Pencil Set (35-Piece)	Andres Bonifacio	2	900.00	2026-10-26
1	Sketch Pencil Set (35-Piece)	Manuel Quezon	1	450.00	2026-10-30

I want to see every individual transaction/sale made for a single specific product (for example, the Sketch Pencil Set (35 Piece)). Since I want details like the customer's name and the sale date along with the product, I'll JOIN the tables and use WHERE to filter for that exact product.

```
SELECT -- Basically asking to show these following.
   p.product_id, -- Creates a column in the virtual table for the id of the products that the customer bought.
   p.product_name, -- Creates a column in the virtual table for the name of the products that the customer bought.
   CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- CONCAT puts the first name and last name together into one column with
                                                               a space in between..
   s.quantity, -- Creates a column in the virtual table for the quantity bought.
   s.total_price, -- Creates a column in the virtual table for the total price of the purchases.
   s.sale_date -- Creates a column in the virtual table for the date of the purchases.
FROM data_customers c -- Sets "c" as the table alias for "data_customers".
JOIN data_sales s ON c.customer_id = s.customer_id -- Sets "c" as the table alias for "data_customers".
JOIN data_products p ON s.product_id = p.product_id -- Sets "p" as the table alias for "data_products", and bridges/matches the product IDs
                                                       from sales and products tables.
WHERE c.product_name = 'Sketch Pencil Set (35-Piece)'; -- Filters the virtual table to only show records for this specific product.
```

TASK 12: Query The Total Sales Of Specific Product

```sql
SELECT
    p.product_id,
    p.product_name,
    SUM(s.quantity) AS total_units_sold,
    SUM(s.total_price) AS total_revenue
FROM data_sales s
JOIN data_products p ON s.product_id = p.product_id
WHERE p.product_name = 'Sketch Pencil Set (35-Piece)'
GROUP BY p.product_id, p.product_name;
```



Showing rows 0 - 0 (1 total, Query took 0.0008 seconds.)

```sql
SELECT p.product_id, p.product_name, SUM(s.quantity) AS total_units_sold, SUM(s.total_price) AS total_revenue FROM data_sales s JOIN data_products p ON s.product_id = p.product_id WHERE p.product_name = 'Sketch Pencil Set (35-Piece)' GROUP BY p.product_id, p.product_name;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

product_id	product_name	total_units_sold	total_revenue
1	Sketch Pencil Set (35-Piece)	10	4500.00

I used SUM() to add up all those sales into a single grand total spent on that item (e.g., Sketch Pencil Set (35-Piece)).

```
SELECT -- Basically asking to show these following.
   p.product_id, -- Creates a column in the virtual table for the id of the products that the customer bought.
   p.product_name, -- Creates a column in the virtual table for the name of the products that the customer bought.
   SUM(s.quantity) AS total_units_sold -- SUM adds up all of the quantity sold into one grand total.
   SUM(s.total_price) AS total_spent -- SUM adds up all of the customer's purchase amounts into one grand total.
FROM data_sales s -- Sets "s" as the table alias for "data_sales".
JOIN data_products p ON s.product_id = p.product_id -- Sets "p" as the table alias for "data_products", and bridges/matches the product IDs
                                                       from sales and products tables.
WHERE c.product_name = 'Sketch Pencil Set (35-Piece)'; -- Filters the virtual table to only show records for this specific product.
GROUP BY s.product_id, s.prodcut_name; -- Gathers all of Sketch Pencil Set's separate purchase receipts and packs them into a single row
                                          so the SUM() function can calculate his overall total.
```

TASK 13: Query The Average Sales Of Specific Product

```sql
SELECT
    p.product_id,
    p.product_name,
    AVG(s.total_price) AS average_sale_amount
FROM data_sales s
JOIN data_products p ON s.product_id = p.product_id
WHERE p.product_name = 'Sketch Pencil Set (35-Piece)'
GROUP BY p.product_id, p.product_name;
```



I used AVG() to find the average transaction amount (or average price paid per order) for that specific item (e.g., Sketch Pencil Set (35-Piece)).

SELECT -- *Basically asking to show these following.*
  p.product_id, -- *Creates a column in the virtual table for the id of the products that the customer bought.*
  p.product_name, -- *Creates a column in the virtual table for the name of the products that the customer bought.*
  AVG(s.total_price) AS average_sale_amount – *Adds up all the total_price then divide by total number, and renames the resulting column header.*
FROM data_sales s -- *Sets "s" as the table alias for "data_sales".*
JOIN data_products p ON s.product_id = p.product_id -- *Sets "p" as the table alias for "data_products", and bridges/matches the product IDs from sales and products tables.*
WHERE c.product_name = 'Sketch Pencil Set (35-Piece)'; -- *Filters the virtual table to only show records for this specific product.*
GROUP BY s.product_id, s.prodcut_name; -- *Gathers all of Sketch Pencil Set's separate purchase receipts and packs them into a single row so the SUM() function can calculate his overall total.*

TASK 14: Query The Top 5 Products Based On Sales

```sql
SELECT
    p.product_id,
    p.product_name,
    SUM(s.quantity) AS total_units_sold,
    SUM(s.total_price) AS total_revenue
FROM data_products p
JOIN data_sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Showing rows 0 - 4 (5 total, Query took 0.0007 seconds.)

```sql
SELECT p.product_id, p.product_name, SUM(s.quantity) AS total_units_sold, SUM(s.total_price) AS total_revenue FROM data_products p JOIN data_sales s ON p.product_id = s.product_id GROUP BY p.product_id, p.product_name ORDER BY total_revenue DESC LIMIT 5;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Extra options

product_id	product_name	total_units_sold	total_revenue ▾ 1
3	Canva Pro Subscription Card	5	7500.00
1	Sketch Pencil Set (35-Piece)	10	4500.00
12	LCD Screen Replacement 6.78"	3	3750.00
11	Scientific Calculator	3	2550.00
2	A4 Bond Paper (500 Sheets)	9	1980.00

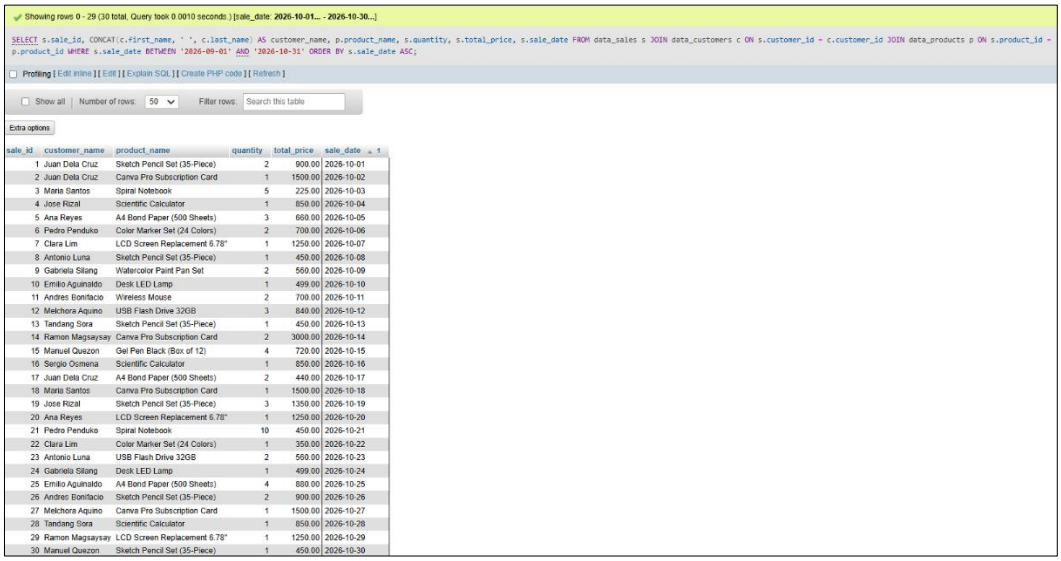
This task combines querying top customers based on sales and query the total sales of specific product. To calculate total revenue for every item using SUM(), group them by product, sort from highest revenue to lowest using ORDER BY ... DESC, and restrict the output to the top 5, using LIMIT 5.

SELECT *-- Basically asking to show these following.*
p.product_id, *-- Creates a column in the virtual table for the id of the products that the customer bought.*
p.product_name, *-- Creates a column in the virtual table for the name of the products that the customer bought.*
SUM(s.quantity) AS total_units_sold *-- SUM adds up all of the quantity sold into one grand total.*
SUM(s.total_price) AS total_revenue *-- SUM adds up all of the customers' purchase amounts into one grand total.*
FROM data_products p *-- Sets "p" as the table alias for "data_products".*
JOIN data_sales s ON p.product_id = s.product_id *-- Sets "p" as the table alias for "data_products", and bridges/matches the product IDs from sales and products tables.*
GROUP BY s.product_id, s.prodcut_name; *-- Gathers all of the products' separate purchase receipts and packs them into a single row so the SUM() function can calculate his overall total.*
ORDER BY total_revenue DESC *– "total_revenue" is going to be displayed in descending order.*
LIMIT 5; *-- It is only going to display a total of 5 results.*

TASK 15: Two Months Sales Queries

By "BETWEEN"

```sql
SELECT
    s.sale_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    p.product_name,
    s.quantity,
    s.total_price,
    s.sale_date
FROM data_sales s
JOIN data_customers c ON s.customer_id = c.customer_id
JOIN data_products p ON s.product_id = p.product_id
WHERE s.sale_date BETWEEN '2026-09-01' AND '2026-10-31'
ORDER BY s.sale_date ASC;
```



Showing rows 0 - 29 (30 total, Query took 0.0010 seconds.) [sale_date: 2026-10-01... - 2026-10-30...]

```
SELECT s.sale_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, p.product_name, s.quantity, s.total_price, s.sale_date FROM data_sales s JOIN data_customers c ON s.customer_id = c.customer_id JOIN data_products p ON s.product_id = p.product_id WHERE s.sale_date BETWEEN '2026-09-01' AND '2026-10-31' ORDER BY s.sale_date ASC;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 | Filter rows: Search this table

Extra options

sale_id	customer_name	product_name	quantity	total_price	sale_date
1	Juan Dela Cruz	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-01
2	Juan Dela Cruz	Canva Pro Subscription Card	1	1500.00	2026-10-02
3	Maria Santos	Spiral Notebook	5	225.00	2026-10-03
4	Jose Rizal	Scientific Calculator	1	850.00	2026-10-04
5	Ana Reyes	A4 Bond Paper (500 Sheets)	3	660.00	2026-10-05
6	Pedro Penduko	Color Marker Set (24 Colors)	2	700.00	2026-10-06
7	Clara Lim	LCD Screen Replacement 6.78"	1	1250.00	2026-10-07
8	Antonio Luna	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-08
9	Gabriela Silang	Watercolor Paint Pan Set	2	560.00	2026-10-09
10	Emilio Aguinaldo	Desk LED Lamp	1	499.00	2026-10-10
11	Andres Bonifacio	Wireless Mouse	2	700.00	2026-10-11
12	Melchora Aquino	USB Flash Drive 32GB	3	840.00	2026-10-12
13	Tandang Sora	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-13
14	Ramon Magsaysay	Canva Pro Subscription Card	2	3000.00	2026-10-14
15	Manuel Quezon	Gel Pen Black (Box of 12)	4	720.00	2026-10-15
16	Sergio Osmena	Scientific Calculator	1	850.00	2026-10-16
17	Juan Dela Cruz	A4 Bond Paper (500 Sheets)	2	440.00	2026-10-17
18	Maria Santos	Canva Pro Subscription Card	1	1500.00	2026-10-18
19	Jose Rizal	Sketch Pencil Set (35-Piece)	3	1350.00	2026-10-19
20	Ana Reyes	LCD Screen Replacement 6.78"	1	1250.00	2026-10-20
21	Pedro Penduko	Spiral Notebook	10	450.00	2026-10-21
22	Clara Lim	Color Marker Set (24 Colors)	1	350.00	2026-10-22
23	Antonio Luna	USB Flash Drive 32GB	2	560.00	2026-10-23
24	Gabriela Silang	Desk LED Lamp	1	499.00	2026-10-24
25	Emilio Aguinaldo	A4 Bond Paper (500 Sheets)	4	880.00	2026-10-25
26	Andres Bonifacio	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-26
27	Melchora Aquino	Canva Pro Subscription Card	1	1500.00	2026-10-27
28	Tandang Sora	Scientific Calculator	1	850.00	2026-10-28
29	Ramon Magsaysay	LCD Screen Replacement 6.78"	1	1250.00	2026-10-29
30	Manuel Quezon	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-30

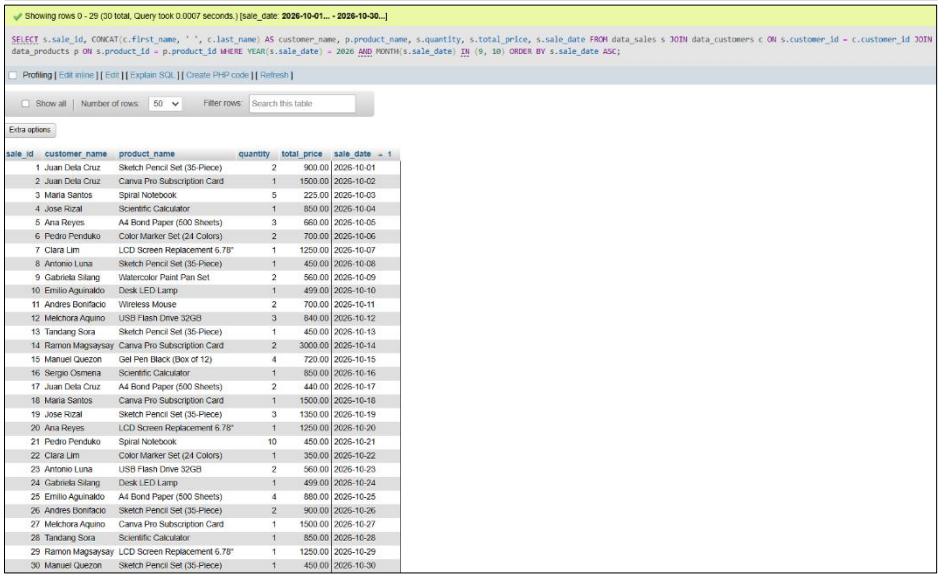
To filter transactions across a specific two-month timeframe (for example, September and October 2026), we use WHERE alongside date filtering functions like BETWEEN or MONTH() & YEAR(), this is how to do it with BETWEEN.

SELECT *-- Basically asking to show these following.*
  s.sale_date *-- Creates a column in the virtual table for the date of the purchases.*
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name, *-- CONCAT puts the first names and last names together into one column with a space in between.*
  p.product_name, *-- Creates a column in the virtual table for the name of the products that the customer bought.*
  s.quantity, *-- Creates a column in the virtual table for the quantity bought.*
  s.total_price, *-- Creates a column in the virtual table for the total price of the purchases.*
  s.sale_date *-- Creates a column in the virtual table for the date of the purchases.*
FROM data_sales s *-- Sets "s" as the table alias for "data_sales".*
JOIN data_customers c ON c.customer_id = c.customer_id *-- Sets "c" as the table alias for "data_customers", and bridges/matches the product IDs from sales and products tables.*
JOIN data_products p ON s.product_id = p.product_id *-- Sets "p" as the table alias for "data_products", and bridges/matches the product IDs from sales and products tables.*
WHERE s.sales_date BETWEEN '2026-09-01' AND 2026-10-31'; *-- Filters the virtual table to only show records for from September 1 to October 31.*
ORDER BY s.sales_date ASC *– It is going to be displayed in ascending order according to their date.*

TASK 15: Two Months Sales Queries

By "MONTH() & YEAR ()"

```sql
SELECT
    s.sale_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    p.product_name,
    s.quantity,
    s.total_price,
    s.sale_date
FROM data_sales s
JOIN data_customers c ON s.customer_id = c.customer_id
JOIN data_products p ON s.product_id = p.product_id
WHERE YEAR(s.sale_date) = 2026
  AND MONTH(s.sale_date) IN (9, 10)
ORDER BY s.sale_date ASC;
```



Showing rows 0 - 29 (30 total, Query took 0.0007 seconds.) [sale_date: 2026-10-01... - 2026-10-30...]

```
SELECT s.sale_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name, p.product_name, s.quantity, s.total_price, s.sale_date FROM data_sales s JOIN data_customers c ON s.customer_id = c.customer_id JOIN
data_products p ON s.product_id = p.product_id WHERE YEAR(s.sale_date) = 2026 AND MONTH(s.sale_date) IN (9, 10) ORDER BY s.sale_date ASC;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 50 | Filter rows: Search this table

Extra options

sale_id	customer_name	product_name	quantity	total_price	sale_date
1	Juan Dela Cruz	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-01
2	Juan Dela Cruz	Canva Pro Subscription Card	1	1500.00	2026-10-02
3	Maria Santos	Spiral Notebook	5	225.00	2026-10-03
4	Jose Rizal	Scientific Calculator	1	850.00	2026-10-04
5	Ana Reyes	A4 Bond Paper (500 Sheets)	3	660.00	2026-10-05
6	Pedro Penduko	Color Marker Set (24 Colors)	2	700.00	2026-10-06
7	Clara Lim	LCD Screen Replacement 6.78"	1	1250.00	2026-10-07
8	Antonio Luna	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-08
9	Gabriela Silang	Watercolor Paint Pan Set	2	560.00	2026-10-09
10	Emilio Aguinaldo	Desk LED Lamp	1	499.00	2026-10-10
11	Andres Bonifacio	Wireless Mouse	2	700.00	2026-10-11
12	Melchora Aquino	USB Flash Drive 32GB	3	840.00	2026-10-12
13	Tandang Sora	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-13
14	Ramon Magsaysay	Canva Pro Subscription Card	2	3000.00	2026-10-14
15	Manuel Quezon	Gel Pen Black (Box of 12)	4	720.00	2026-10-15
16	Sergio Osmeña	Scientific Calculator	1	850.00	2026-10-16
17	Juan Dela Cruz	A4 Bond Paper (500 Sheets)	2	440.00	2026-10-17
18	Maria Santos	Canva Pro Subscription Card	1	1500.00	2026-10-18
19	Jose Rizal	Sketch Pencil Set (35-Piece)	3	1350.00	2026-10-19
20	Ana Reyes	LCD Screen Replacement 6.78"	1	1250.00	2026-10-20
21	Pedro Penduko	Spiral Notebook	10	450.00	2026-10-21
22	Clara Lim	Color Marker Set (24 Colors)	1	350.00	2026-10-22
23	Antonio Luna	USB Flash Drive 32GB	2	560.00	2026-10-23
24	Gabriela Silang	Desk LED Lamp	1	499.00	2026-10-24
25	Emilio Aguinaldo	A4 Bond Paper (500 Sheets)	4	880.00	2026-10-25
26	Andres Bonifacio	Sketch Pencil Set (35-Piece)	2	900.00	2026-10-26
27	Melchora Aquino	Canva Pro Subscription Card	1	1500.00	2026-10-27
28	Tandang Sora	Scientific Calculator	1	850.00	2026-10-28
29	Ramon Magsaysay	LCD Screen Replacement 6.78"	1	1250.00	2026-10-29
30	Manuel Quezon	Sketch Pencil Set (35-Piece)	1	450.00	2026-10-30

To filter transactions across a specific two-month timeframe (for example, September and October 2026), we use WHERE alongside date filtering functions like BETWEEN or MONTH() & YEAR(), this is how to do it with MONTH() & YEAR.

SELECT -- *Basically asking to show these following.*
  s.sale_date -- *Creates a column in the virtual table for the date of the purchases.*
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name, -- *CONCAT puts the first names and last names together into one column with a space in between.*
  p.product_name, -- *Creates a column in the virtual table for the name of the products that the customer bought.*
  s.quantity, -- *Creates a column in the virtual table for the quantity bought.*
  s.total_price, -- *Creates a column in the virtual table for the total price of the purchases.*
  s.sale_date -- *Creates a column in the virtual table for the date of the purchases.*
FROM data_sales s -- *Sets "s" as the table alias for "data_sales".*
JOIN data_customers c ON c.customer_id = c.customer_id -- *Sets "c" as the table alias for "data_customers", and bridges/matches the product IDs from sales and products tables.*
JOIN data_products p ON s.product_id = p.product_id -- *Sets "p" as the table alias for "data_products", and bridges/matches the product IDs from sales and products tables.*
WHERE YEAR(s.sale_date) = 2026 – *Filters the year of the results, which is 2026.*
  AND MONTH(s.sale_date) IN (9, 10) – *Filters the months of the results, which is September to October.*
ORDER BY s.sales_date ASC – *It is going to be displayed in ascending order according to their date.*